

Building a Simple Agent Framework 2

Now, we're going to put the components together into a usable agent class. This class will encapsulate the SBDR components and provide a simple interface for running the agent loop. The agent will be responsible for constructing prompts, extracting actions, and managing memory. We can create different agents simply by changing the goals, actions, and environment without modifying the core logic.

```
class SimpleAgent:
    """
    A simple agent class that encapsulates the SBDR components.
    """
    def __init__(
        self,
        goals: List[str],
        actions: List[str],
        memory: List[str],
        environment: str,
        verbose: bool = False,
    ):
        """
        Initialize the agent with the core SBDR components.
        """
        self.goals = goals
        self.actions = actions
        self.memory = memory
        self.environment = environment
        self.verbose = verbose

    def _construct_prompt(self, goals: List[str], memory: List[str], environment: str) -> str:
        """
        Construct a prompt with memory, environment, and goals.
        """
        prompt = f"Memory: {memory}\nEnvironment: {environment}\nGoals: {goals}"
        return prompt

    def _extract_actions(self, response: str) -> List[str]:
        """
        Extract actions from the response.
        """
        actions = []
        for action in response.split("\n"):
            if action.startswith("Action:"):
                actions.append(action.strip().replace("Action:", ""))
        return actions

    def _update_memory(self, response: str, actions: List[str]) -> List[str]:
        """
        Update memory with the response and actions.
        """
        new_memory = []
        for line in response.split("\n"):
            if line not in self.memory:
                new_memory.append(line)
        for action in actions:
            new_memory.append(f"Action: {action}")
        return new_memory

    def _run_loop(self, max_iterations: int = 10):
        """
        Run the agent loop.
        """
        prompt = self._construct_prompt(self.goals, self.memory, self.environment)
        response = self._generate_response(prompt)
        actions = self._extract_actions(response)
        self.memory = self._update_memory(response, actions)
        return response, actions
```

Now, we'll walk through how the SBDR components work together in the agent architecture, exploring each part of agent loop.

Step 1: Constructing the Prompt

When the agent bootstraps, it'll construct a prompt using the `construct_prompt` method.

```
def construct_prompt(
    goals: List[str],
    memory: List[str],
    environment: str,
) -> str:
    """
    Construct a prompt with memory, environment, and goals.
    """
    prompt = f"Memory: {memory}\nEnvironment: {environment}\nGoals: {goals}"
    return prompt
```

The method constructs the `construct_prompt` component to build a structured prompt containing:

- The agent's goals (what it's trying to accomplish)
- A list of actions (what the agent can do)
- Current memory context (previous actions, history, and relevant information)
- The environment (what the agent is operating in)

We are going to discuss the `construct_prompt` in more detail later. For now, what you need to know is that it is responsible for formatting the prompt that is sent to the LLM and parsing the response from the LLM. Most of the time, we are going to use function calling in the parsing step just to make the returned tool calls. However, the `construct_prompt` can be changed to allow us to also take the same agent and implement it without function calling.

Step 2: Generating a Response

Now, the agent sends the prompt to the language model.

```
def generate_response(
    prompt: str,
) -> str:
    """
    Generate a response from the language model.
    """
    response = llm.generate(prompt)
    return response
```

The `generate_response` function is a simple placeholder provided during initialization. This abstraction allows the framework to work with different language models without changing the core logic. We will use `OpenAI` to call the LLM, but you could easily swap this out for any other LLM provider.

Step 3: Parsing the Response

Once the language model returns a response, the agent needs to identify the intended action. The parsing step generally just gets the tool calls from the response. However, the agent's language goals to decide how this is done. Once the response is parsed, the agent can look at the action in the `extract_actions`.

```
def extract_actions(
    response: str,
) -> List[str]:
    """
    Extract actions from the response.
    """
    actions = []
    for action in response.split("\n"):
        if action.startswith("Action:"):
            actions.append(action.strip().replace("Action:", ""))
    return actions
```

The `extract_actions` is the core logic of what the agent "sees" as the `extract_actions` is the specific parameters that the agent has chosen to use for the action. The `extract_actions` allows the agent to look up the action by name, and the `extract_actions` provides the arguments needed to execute it. This could also be added at this step to ensure that the extraction process matches the agent's required parameters.

Step 4: Executing the Action

The agent then executes the action within the environment.

```
def execute_action(
    action: str,
) -> str:
    """
    Execute the action in the environment.
    """
    result = environment.execute(action)
    return result
```

The `execute_action` handles the actual execution of the action, which might involve:

- Calling APIs
- Interacting with files
- Performing calculations
- Returning data

Actions are defined in the `extract_actions` but executed within the context of the `execute_action`, which provides access to resources and handles the mechanics of execution.

Step 5: Updating Memory

We'll maintain the agent's memory with both its history and the result.

```
def update_memory(
    memory: List[str],
    response: str,
    actions: List[str],
) -> List[str]:
    """
    Update memory with the response and actions.
    """
    new_memory = []
    for line in response.split("\n"):
        if line not in memory:
            new_memory.append(line)
    for action in actions:
        new_memory.append(f"Action: {action}")
    return new_memory
```

This creates a continuous record of the agent's responses and actions, which becomes part of the context of future loop iterations. The memory serves both as a record of past actions and as context for future prompt construction.

Step 6: Termination Check

Finally, the agent checks if it should terminate the loop.

```
def should_terminate(
    memory: List[str],
    actions: List[str],
) -> bool:
    """
    Check if the agent should terminate.
    """
    return False
```

This abstraction allows (like a "terminal" action) to signal that the agent has finished its work.

The Flow of Information Through the Loop

To better understand how these components interact, we'll trace how information flows through a single iteration of the loop.

- The `bootstrap` provides context about what the agent has started to do in its environment and results from the agent loop.
- The `construct_prompt` defines what the agent is trying to accomplish and what it has to work with.
- The `generate_response` sends the prompt to the LLM and receives the response, which will typically be structured to contain tool calls.
- The `extract_actions` identifies the actions within the response, which will typically be structured to contain tool calls.
- The `execute_action` performs the actions within the environment, which will typically be structured to contain tool calls.
- The `update_memory` maintains the agent's memory with the response and actions.
- The loop repeats with the updated memory and the agent calls external tools to resolve the maximum number of iterations.

Creating Specialized Agents

The beauty of this framework is that we can create entirely different agents by changing the SBDR components without modifying the core logic.

```
# Example 1: A simple agent
def simple_agent():
    goals = ["Find the weather in New York"]
    actions = ["get_weather", "search_web"]
    memory = []
    environment = SimpleEnvironment()
    agent = SimpleAgent(goals, actions, memory, environment)
    return agent

# Example 2: A more complex agent
def complex_agent():
    goals = ["Analyze the stock market trends for Tech stocks"]
    actions = ["get_stock_prices", "analyze_trends", "search_news"]
    memory = []
    environment = ComplexEnvironment()
    agent = SimpleAgent(goals, actions, memory, environment)
    return agent
```

Each agent operates using the same fundamental logic but handles completely different tasks based on its SBDR components.